

Lecture # 06

Concurrency Controls (Mutex & Semaphore)

PARALLEL AND DISTRIBUTED COMPUTING

CS-482



Introduction

In the Operating System, **Mutex and Semaphores** are kernel resources that provide synchronization services (also known as synchronization primitives). Synchronization is required when multiple processes are executing concurrently, to avoid conflicts between processes using shared resources. In this article we will see differences between Mutex and Semaphore, their advantages and disadvantages.

What is Mutex?

A mutex is different from a binary semaphore, which provides a locking mechanism. It stands for Mutual Exclusion Object. Mutex is mainly used to provide mutual exclusion to a specific portion of the code so that the process can execute and work with a particular section of the code at a particular time. A mutex enforces strict ownership. Only the thread that locks the mutex can unlock it. It is specifically used for locking a resource to ensure that only one thread accesses it at a time. Due to this strict ownership, a mutex is not only typically used for signaling between threads, but it is used for mutual exclusion also to ensuring that a resource is accessed by only one thread at a time.

Mutex uses a priority inheritance mechanism to avoid priority inversion issues. The priority inheritance mechanism keeps higher-priority processes in the blocked state for the minimum possible time. However, this cannot avoid the priority inversion problem, but it can reduce its effect up to an extent.

Mutual Exclusion in Synchronization

During concurrent execution of processes, processes need to enter the critical section (or the section of the program shared across processes) at times for execution. It might happen that because of the execution of multiple processes at once, the values stored in the critical section become inconsistent. In other words, the values depend on the sequence of execution of instructions – also known as a race condition. The primary task of process synchronization is to get rid of race conditions while executing the critical section.

What is Mutual Exclusion?

Mutual Exclusion is a property of process synchronization that states that “**no two processes can exist in the critical section at any given point of time**”. The term was first coined by **Dijkstra**. Any process synchronization technique being used must satisfy the property of mutual exclusion, without which it would not be possible to get rid of a race condition.

The need for mutual exclusion comes with concurrency. There are several kinds of concurrent execution:

- Interrupt handlers
- Interleaved, preemptively scheduled processes/threads
- Multiprocessor clusters, with shared memory
- Distributed systems

Mutual exclusion methods are used in concurrent programming to avoid the simultaneous use of a common resource, such as a global variable, by pieces of computer code called critical sections.

The requirement of mutual exclusion is that when process P1 is accessing a shared resource R1, another process should not be able to access resource R1 until process P1 has finished its operation with resource R1.

Examples of such resources include files, I/O devices such as printers, and shared data structures.

Conditions Required for Mutual Exclusion

According to the following four criteria, mutual exclusion is applicable:

- When using shared resources, it is important to ensure mutual exclusion between various processes. There cannot be two processes running simultaneously in either of their critical sections.
- It is not advisable to make assumptions about the relative speeds of the unstable processes.
- For access to the critical section, a process that is outside of it must not obstruct another process.
- Its critical section must be accessible by multiple processes in a finite amount of time; multiple processes should never be kept waiting in an infinite loop.

Approaches To Implementing Mutual Exclusion

- **Software Method:** Leave the responsibility to the processes themselves. These methods are usually highly error-prone and carry high overheads.
- **Hardware Method:** Special-purpose [machine instructions](#) are used for accessing shared resources. This method is faster but cannot provide a complete solution. Hardware solutions cannot give guarantee the absence of [deadlock](#) and [starvation](#).
- **Programming Language Method:** Provide support through the operating system or through the programming language.

Requirements of Mutual Exclusion

- At any time, only one process is allowed to enter its critical section.
- The solution is implemented purely in software on a machine.
- A process remains inside its critical section for a bounded time only.
- No assumption can be made about the relative speeds of asynchronous concurrent processes.
- A process cannot prevent any other process from entering into a critical section.
- A process must not be indefinitely postponed from entering its critical section.

Semaphores in Process Synchronization

Semaphores are a tool used in operating systems to help manage how different processes (or programs) share resources, like memory or data, without causing conflicts. A semaphore is a special kind of synchronization data that can be used only through specific synchronization primitives. Semaphores are used to implement critical sections, which are regions of code that must be executed by only one process at a time. By using semaphores, processes can coordinate access to shared resources, such as shared memory or I/O devices.

What is Semaphores?

A semaphore is a synchronization tool used in concurrent programming to manage access to shared resources. It is a lock-based mechanism designed to achieve process synchronization, built on top of basic locking techniques.

Semaphores use a counter to control access, allowing synchronization for multiple instances of a resource. Processes can attempt to access one instance, and if it is not available, they can try other instances. Unlike basic locks, which allow only one process to access one instance of a resource. Semaphores can handle more complex synchronization scenarios, involving multiple processes or threads. It help prevent problems like **race conditions** by controlling when and how processes access shared data.

The process of using Semaphores provides two operations:

- **wait (P):** The wait operation decrements the value of the semaphore
- **signal (V):** The signal operation increments the value of the semaphore.

When the value of the semaphore is zero, any process that performs a wait operation will be blocked until another process performs a signal operation.

When a process performs a wait operation on a semaphore, the operation checks whether the value of the semaphore is >0 . If so, it decrements the value of the semaphore and lets the process continue its execution; otherwise, it blocks the process on the semaphore. A signal operation on a semaphore activates a process blocked on the semaphore if any, or increments the value of the semaphore by 1. Due to these semantics, semaphores are also called counting semaphores. The initial value of a semaphore determines how many processes can get past the wait operation.

Semaphores are required for process synchronization to make sure that multiple processes can safely share resources without interfering with each other. They help control when a process can access a shared resource, preventing issues like **race** conditions.

Types of Semaphores

Semaphores are of two Types:

- **Binary Semaphore:** This is also known as a mutex lock, as they are locks that provide mutual exclusion. It can have only two values – 0 and 1. Its value is initialized to 1. It is used to implement the solution of critical section problems with multiple processes and a single resource.
- **Counting Semaphore:** Counting semaphores can be used to control access to a given resource consisting of a finite number of instances. The semaphore is initialized to the number of resources available. Its value can range over an unrestricted domain.

Working of Semaphore

A semaphore is a simple yet powerful synchronization tool used to manage access to shared resources in a system with multiple processes. It works by maintaining a counter that controls access to a specific resource, ensuring that no more than the allowed number of processes access the resource at the same time.

There are two primary operations that a semaphore can perform:

- 1.Wait (P operation):** This operation checks the semaphore's value. If the value is greater than 0, the process is allowed to continue, and the semaphore's value is decremented by 1. If the value is 0, the process is blocked (waits) until the semaphore value becomes greater than 0.
- 2.Signal (V operation):** After a process is done using the shared resource, it performs the signal operation. This increments the semaphore's value by 1, potentially unblocking other waiting processes and allowing them to access the resource.

Working of Semaphore

```
P(Semaphore s){  
    while (S == 0); /* wait until s = 0 */  
    s = s - 1;  
}
```

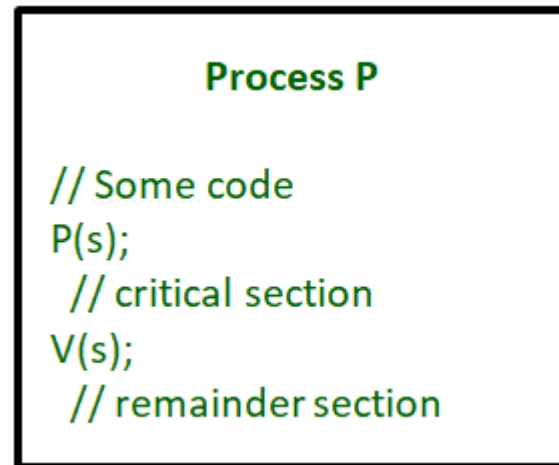
```
V(Semaphore s){  
    s = s + 1;  
}
```



Note that there is
Semicolon after while.
The code gets stuck
Here while s is 0.

Working of Semaphore

A critical section is surrounded by both operations to implement process synchronization. The below image demonstrates the basic mechanism of how semaphores are used to control access to a critical section in a multi-process environment, ensuring that only one process can access the shared resource at a time

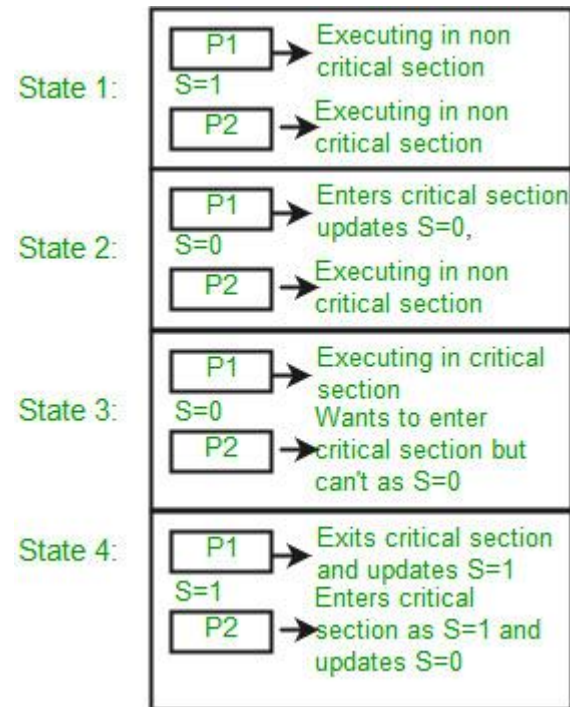


Working of Semaphore

Now, let us see how it implements mutual exclusion. Let there be two processes P1 and P2 and a semaphore s is initialized as 1. Now if suppose P1 enters in its critical section then the value of semaphore s becomes 0. Now if P2 wants to enter its critical section then it will wait until $s > 0$, this can only happen when P1 finishes its critical section and calls V operation on semaphore s .

This way mutual exclusion is achieved. Look at the below image for details which is a Binary semaphore.

Working of Semaphore



Working of Binary Semaphore

```
P(semaphore s)
{
    if (s.value == 1) {
        s.value = 0;
    }
    else {
        // add the process to the waiting queue
        q.push(P) sleep();
    }
}

V(semaphore s)
{
    if (s.q is empty) {
        s.value = 1;
    }
    else {
        // select a process from waiting queue
        Process p = q.front();
        // remove the process from waiting as it has been
        // sent for CS
        q.pop();
        wakeup(p);
    }
}
```

Working of Binary Semaphore

The description above is for binary semaphore which can take only two values 0 and 1 and ensure mutual exclusion. There is one other type of semaphore called counting semaphore which can take values greater than one.

Now suppose there is a resource whose number of instances is 4. Now we initialize $S = 4$ and the rest is the same as for binary semaphore. Whenever the process wants that resource, it calls P or waits for function and when it is done it calls V or signal function. If the value of S becomes zero then a process has to wait until S becomes positive. For example, Suppose there are 4 processes P1, P2, P3, and P4, and they all call wait operation on S(initialized with 4). If another process P5 wants the resource then it should wait until one of the four processes calls the signal function and the value of semaphore becomes positive.

Limitations

- One of the biggest limitations of semaphore is priority inversions.
- Deadlock, suppose a process is trying to wake up another process that is not in a sleep state. Therefore, a deadlock may be blocked indefinitely.
- The operating system has to keep track of all calls to wait and signal the semaphore.

The main problem with semaphores is that they require busy waiting, If a process is in the critical section, then other processes trying to enter the critical section will be waiting until the critical section is not occupied by any process. Whenever any process waits then it continuously checks for semaphore value (look at this line while (s==0); in P operation) and wastes CPU cycle.

There is also a chance of “spinlock” as the processes keep on spinning while waiting for the lock. To avoid this another implementation is provided below.

Updated

```
P(Semaphore s)
{
    s.value = s.value - 1;
    if (s.value < 0) {
        // add process to queue
        // here p is a process which is currently executing
        q.push(p);
        block();
    }
    else
        return;
}
```

```
V(Semaphore s)
{
    s.value = s.value + 1;
    if (s.value <= 0) {
        // remove process p from queue
        Process p = q.pop();
        wakeup(p);
    }
    else
        return;
}
```

Explanation of Code

In this implementation whenever the process waits it is added to a waiting queue of processes associated with that semaphore. This is done through the system call `block()` on that process. When a process is completed, it calls the signal function and one process in the queue is resumed. It uses the `wakeup()` system call.

Uses of Semaphores

- **Mutual Exclusion** : Semaphore ensures that only one process accesses a shared resource at a time.
- **Process Synchronization** : Semaphore coordinates the execution order of multiple processes.
- **Resource Management** : Limits access to a finite set of resources, like printers, devices, etc.
- **Reader-Writer Problem** : Allows multiple readers but restricts the writers until no reader is present.
- **Avoiding Deadlocks** : Prevents deadlocks by controlling the order of allocation of resources.

Advantages of Semaphores

- Semaphore is a simple and effective mechanism for process synchronization
- Supports coordination between multiple processes. By controlling the access to critical sections, semaphores help in managing multiple processes without them interfering with each other.
- When used correctly, semaphores can help avoid deadlocks by managing access to resources efficiently and ensuring that no process is indefinitely blocked from accessing necessary resources.
- Semaphores help prevent race conditions by ensuring that only one process can access a shared resource at a time.
- Provides a flexible and robust way to manage shared resources.

Disadvantages of Semaphores

- It Can lead to performance degradation due to overhead associated with wait and signal operations.
- If semaphores are not managed carefully, they can lead to deadlock. This often occurs when semaphores are not released properly or when processes acquire semaphores in an inconsistent order.
- It can cause performance issues in a program if not used properly.
- It can be difficult to debug and maintain. Debugging systems that rely heavily on semaphores can be challenging, as it is hard to track the state of each semaphore and ensure that all processes are correctly synchronized
- It can be prone to race conditions and other synchronization problems if not used correctly.
- It can be vulnerable to certain types of attacks, such as denial of service attacks.